

InternStellar Architecture Overview

EECS 582, Computer Science Capstone

Group 23: Navya Nittala, Kundana Dongala, Katie Nordberg, Vivian Lara, Christina Sorensen

Project Name: InternStellar

Project Synopsis: A 2D space adventure game where a NASA intern solves puzzles across planets to uncover secrets and save Earth from alien invasion.

Architecture Description:

- **Overview:** InternStellar is a 2D point-and-click adventure game developed using the Godot Engine and the Godot Script programming language. The software architecture follows a modular, scene-based design that separates gameplay systems into independent but interconnected components. The primary goal of this architecture is to maximize scalability, maintainability, and ease of expansion while preserving clean system boundaries. This will allow us to expand the game by creating new levels easily.

The architecture is divided into five major subsystems:

- Game Core/State Management
- Interaction System
- UI System
- Navigation & Map System
- Puzzle & Story Modules

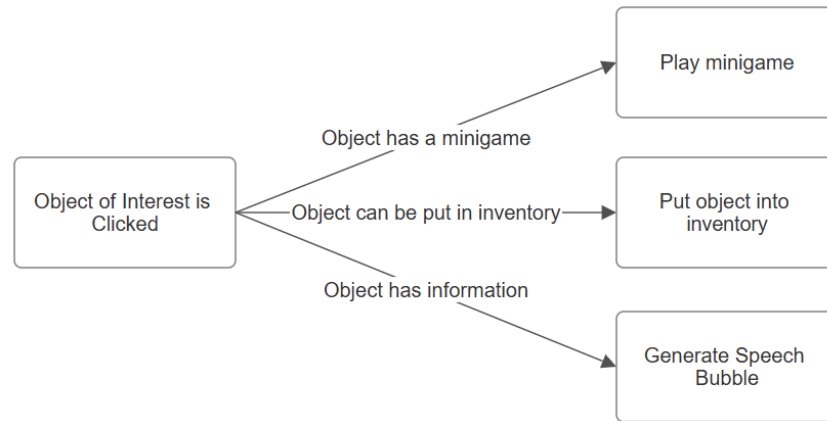
Each subsystem communicates through an event-driven signal mechanism provided by Godot. This prevents tight coupling between modules and allows components to evolve independently.

The GameManager acts as the central authority for the global state. It tracks:

- Player inventory
- Puzzle completion states
- Story progression flags
- Unlocked planets

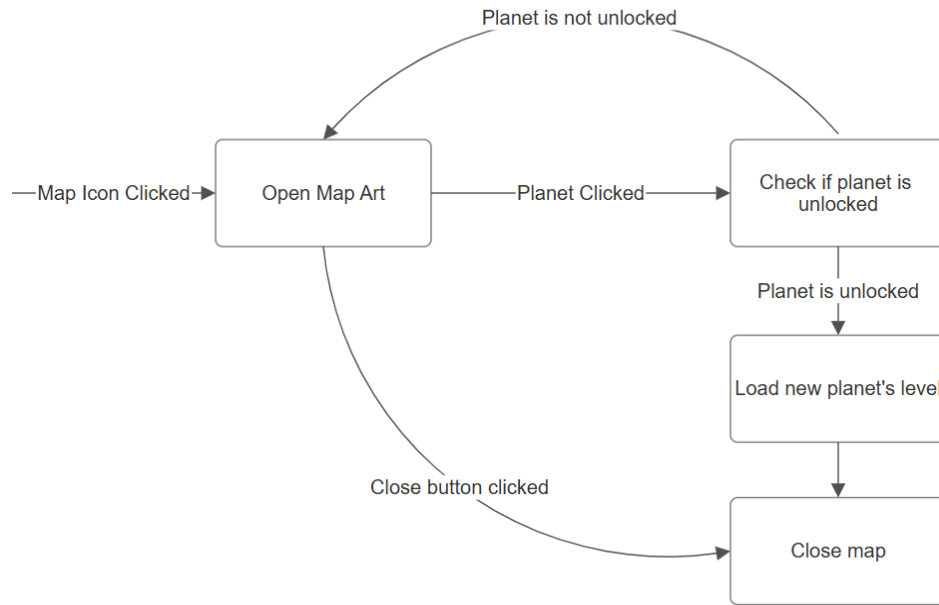
By centralizing state management, InternStellar ensures consistency across scenes and avoids duplicated logic.

- **Point and click functionality:** The player will click on an object of interest. This will interact with the object by either placing it in the player's inventory, generating a speech bubble, or starting a mini game.

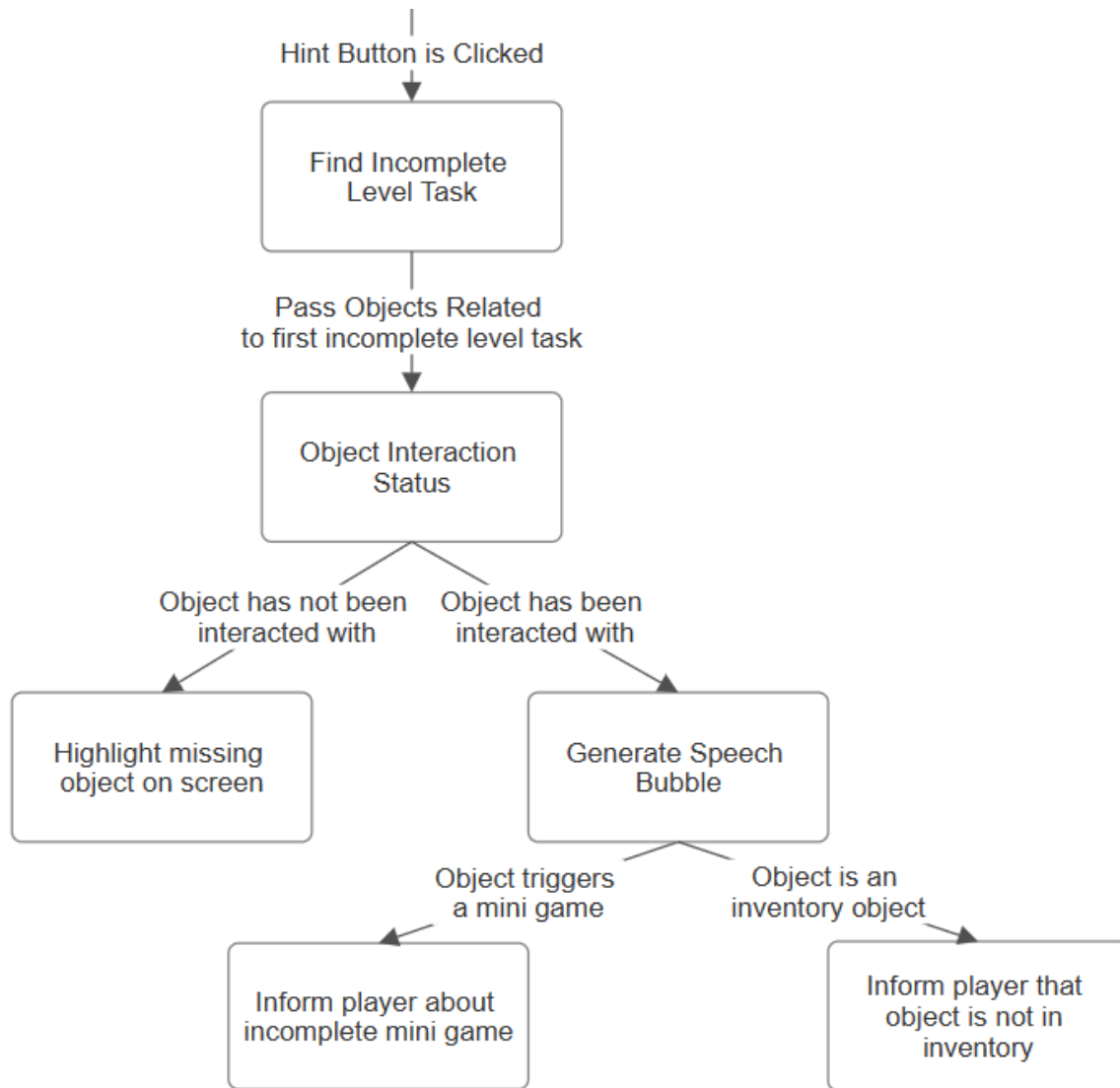


When the player clicks on an object, the system first detects which object was selected. Each object in the game is assigned properties that determine how it should respond to interaction.

- If the object has a minigame, the game transitions into that activity, allowing the player to complete a small challenge or puzzle relevant to the clicked object. For instance, in level one, clicking on a disorganized bookshelf will launch a minigame in which the player must sort the books.
- If the object can be put in the inventory, it is collected and stored for later use in the game. For example, in level one, completing a puzzle drops a key that can be put in the player's inventory and used to unlock a desk drawer.
- If the object has information, the game displays a speech bubble containing relevant dialogue, hints, or story text.
- **Map for navigation:** The player can use their map to navigate to new planets or rooms. Planets and other checkpoints will be unlocked as the game progresses. If a checkpoint is clicked and unlocked, the checkpoint's corresponding level will be loaded, and the map view will be closed.



- **Hint system:** The player can get a hint when they are stuck by clicking on the hint button. When this happens, the game will determine what tasks are still incomplete. All objects related to the first incomplete task will be examined to see if the user has clicked on them yet. If an object exists that has never been clicked on, the game will highlight it. If every object has been clicked on, then the game will generate a speech bubble explaining what action still needs to be taken (e.g. either a mini game needs to be won or an object needs to be added to the player's inventory).

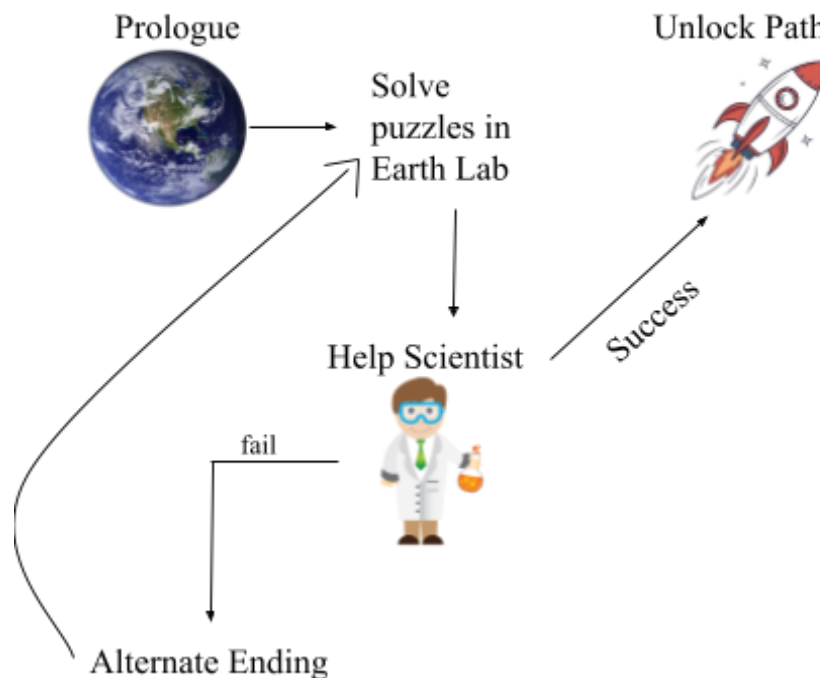


System Components:

- **Point and click**
 - Allows the player to click on the objects
 - Clicking will trigger one of three outcomes:
 - A speech bubble providing clues or story exposition will appear
 - A mini game or puzzle will be launched
 - The object will be put into the player's inventory
- **Game engine/core systems: Godot**
 - Manages rendering, input, scene transitions, and game state
 - Maintains Player inventory, puzzle states, and unlocked locations
- **UI**
 - The UI system uses a CanvasLayer with reusable components:

- Inventory Panel: Displays collected items and allows item usage.
 - Dialogue Box: Shows speech bubbles with text and choices.
 - Map Screen: Displays unlocked planets and checkpoints that can be used for navigation.
- The UI connects with other systems through Godot signals. For example, when a puzzle is completed, the UI updates to show newly unlocked areas or collected resources.
- [Map navigation system](#)
 - Updates accessible areas based on the story progression

Navigation and Story Progression



- [Story module](#)
 - Provides context and game lore, introduces new challenges, and guides the player on what their next action should be
 - Supports branching dialogue and minor player choices that influence gameplay
- [Mini-game/Puzzle](#)
 - Pipe puzzles (e.g. connect the pipes to allow the water to go through)
 - Determine what materials work together to create something
 - Untangle wires
 - Determine the pattern
 - Complete a Nonogram (e.g. to form a picture of something we need)
 - Cleaning a sample (e.g. like a special mineral or something surrounded by rock)

- Translate the ancient language
- Find the password for a locked computer
- Enter the launch codes into a computer terminal

Project Description:

At its core, our project is an engaging and challenging but fun 2D point-and-click video game. Our game takes place both on Earth and throughout outer space. The concept of this game is a familiar one - there is an alien invasion threatening to take over planet Earth. But the twist is that the only person who can prevent this impending doom and save the planet is one of NASA's newest interns. The players will explore different locations throughout the galaxy, solve puzzles, and uncover secrets that will reveal how to save Earth and humanity as we know it.

How to Play:

The game starts with a cutscene that introduces the story and main character. After the cutscene, the player will start on level one, where they will solve a few puzzles to figure out why their team is missing. Once they discover their team has been abducted, they must find the launch codes via a series of puzzles and clues. Each level will play out in a similar manner of having the player complete tasks to advance. Each level plays like an escape room, requiring players to explore their surroundings, collect useful items, and piece together clues to progress. As players solve challenges, new areas and planets become unlocked, moving the story forward and bringing them closer to saving Earth.

Conclusion

InternStellar's architecture balances creative gameplay with structured engineering principles. By separating core systems like interaction, UI, puzzles, story, and navigation, the game achieves extensibility and maintainability appropriate for a capstone-level software project.

The modular design ensures future expansion while preserving system stability. New puzzles, planets, and story arcs can be added without rewriting foundational systems.

This architecture supports iterative development, team collaboration, and long-term scalability while delivering an engaging point-and-click adventure experience.